

Лекция 6

Обучение с подкреплением

Никита Юдин, iudin.ne@phystech.edu

Московский физико-технический институт
Физтех-школа прикладной математики и информатики

12 марта 2026



Что делали?

Q-функция по политике π

$$Q^\pi(s, a) = \mathbb{E}_\tau \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] = \mathbb{E}_\tau [Z^\pi(s, a)],$$

где $\tau \sim p(s_0) \cdot \prod_{t=0}^{\infty} (\pi(a_t | s_t) p(s_{t+1} | s_t, a_t))$. До этого момента вопрос о распределении Z^π нас несильно интересовал — мы искали сразу её математическое ожидание.

Что делали?

Идея

Почему бы нам не искать распределение Z^π (т.е. распределение наград) и понимать о задаче больше?

Иными словами заставим нейронную сеть аппроксимировать функцию Z и будем выбирать оптимальное действие согласно

$$a^* = \arg \max_{a \in \mathcal{A}} \{\mathbb{E} [Z^\pi(s, a)]\}.$$

Вопрос к залу

Каким образом заставить нейросеть возвращать распределение?

Что делали?

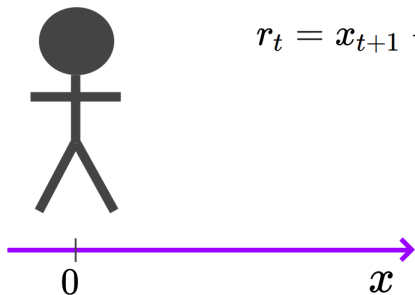
QR-DQN

Зафиксируем N и будем генерировать N дельта-функций. Их позиции будет выбирать нейронная сеть, каждая дельта-функция принимает максимальное значение $\frac{1}{N}$.

Следующий вопрос

Каким образом учить такую сеть? — Смотри предыдущую лекцию.

Жадность — плохое качество

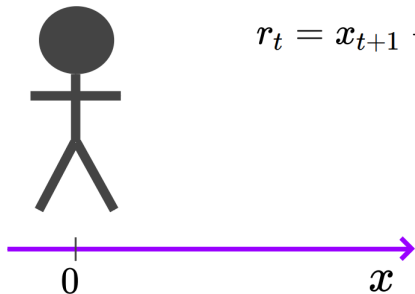


$$r_t = x_{t+1} - x_t$$

Жадная политика агента:

- упасть — маленькая награда
 $r_{\text{fall}} = \Delta x;$

Жадность — плохое качество

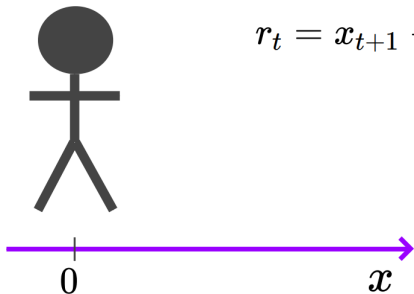


$$r_t = x_{t+1} - x_t$$

Жадная политика агента:

- упасть — маленькая награда $r_{\text{fall}} = \Delta x$;
- идти вперед — награда пропорциональна движению;

Жадность — плохое качество



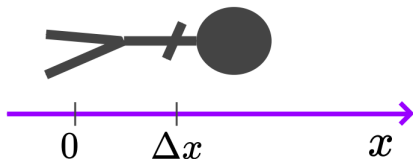
$$r_t = x_{t+1} - x_t$$

Жадная политика агента:

- упасть — маленькая награда $r_{\text{fall}} = \Delta x$;
- идти вперед — награда пропорциональна движению;
- init оценка Q -функции:
 $Q(s_0, a_{\text{fall}}) = 0$;
 $Q(s_0, a_{\text{step}}) = 0$.

Жадность — плохое качество

$$r_t = x_{t+1} - x_t$$

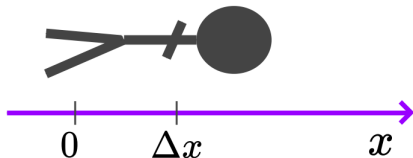


Жадная политика агента:

- если политика жадная и нам не повезло выбрать первым действием падение, то награда всегда будет Δx , потому что агент не исследует!

Жадность — плохое качество

$$r_t = x_{t+1} - x_t$$



Жадная политика агента:

- если политика жадная и нам не повезло выбрать первым действием падение, то награда всегда будет Δx , потому что агент не исследует!
- Вывод: нужно добавить *exploration*, подобно тому как мы делали это в *DQN*.

Одно «но»

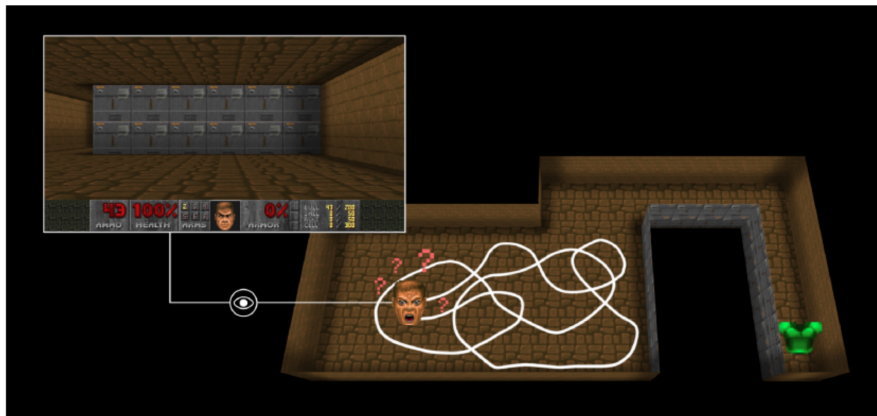
Внимание

Несмотря на то что для DQN с ε -жадной стратегией есть целая теорема о сходимости, на практике сходимость может быть ужасно плохой. Это связано прежде всего с тем, что в теореме используется *бесконечное количество взаимодействий со средой*.

Представьте

А что если у нас среда — трехмерный DOOM????

Как я буду это исследовать?



Среды с разреженной, малоинформативной наградой

Пример

Задачей поиска будем называть задачу со следующей функцией награды:

$$r(s, a) = \begin{cases} +1 & s \in \mathcal{S}^+; \\ \text{const} & s \notin \mathcal{S}^+, \end{cases}$$

где $\mathcal{S}^+$ — множество терминальных состояний, $\text{const} \leq 0$ — штраф за потерю времени.

Напоминание

Марковский процесс принятия решений — кортеж из $(\mathcal{S}, \mathcal{A}, r, \mathcal{T}, \gamma)$, где:

$\mathcal{A}$ — множество действий,

$\mathcal{S}$ — множество состояний,

$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ — функция наград,

$\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ — функция вероятности перехода,

для которого $\mathcal{T}(s, a, s') = \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$, то есть вероятность оказаться в следующем состоянии зависит только от предыдущего состояния и действия из него.

Для решения нашей проблемы с *exploration* попробуем изменить функцию награды r .

Self-Supervision

Markov decision process:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ — функция вероятности перехода;
- $r^{\text{extr}} : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ — функция наград для исходной задачи (внешняя).

Self-Supervision

Markov decision process:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ — функция вероятности перехода;
- $r^{\text{extr}} : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ — функция наград для исходной задачи (внешняя).

Auxiliary task:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
- $\mathcal{S} \times \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ — функция вероятности перехода;
- $r^{\text{intr}} : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ — функция наград для лучшего *exploration* (внутренняя).

Self-Supervision

Аддитивная мотивация

$$r(s, a) := r^{\text{extr}}(s, a) + \alpha r^{\text{intr}}(s, a),$$

где α — масштабирующий гиперпараметр. Для простоты далее будем считать $\alpha = 1$.

Утверждение

$$Q^\pi(s, a) = Q_{\text{extr}}^\pi(s, a) + Q_{\text{intr}}^\pi(s, a);$$

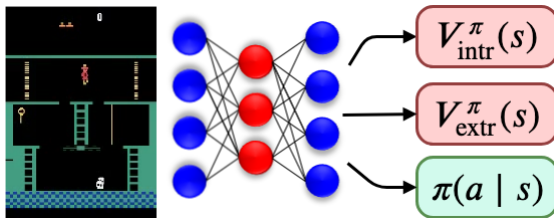
$$V^\pi(s) = V_{\text{extr}}^\pi(s) + V_{\text{intr}}^\pi(s).$$

Доказательство:

по определению. □

Self-Supervision

Если в алгоритме учиться оценочная функция V^π или Q^π , то можно учить оценочные функции каждого слагаемого в награде по отдельности, например, в виде отдельной головы для каждой мотивации.



Трюк можно применять даже в value-based методах, где мы учим Q^* , поскольку тот процесс можно интерпретировать как оценивание (обучение Q^π) текущей стратегии $\pi(s) = \arg \max_{a \in \mathcal{A}} \{Q^\pi(s, a)\}$.

Self-Supervision

Утверждение

В общем случае аналогичное разложение для Q^*, V^* неверно.

Доказательство:

Максимум суммы может быть меньше суммы максимумов.

Действительно, пусть первое действие даёт внешнюю $+1$, а второе действие даёт внутреннюю $+1$ (иначе по нулям); тогда оптимальные оценочные функции равны $V_{\text{intr}}^*(s) = V_{\text{extr}}^*(s) = +1$, но оптимальная V -функция для суммы наград равна не 2, а 1, поскольку выбрать одновременно два действия нельзя и между мотивациями придётся выбирать. □

В Distributional-подходе при использовании этой идеи придётся предполагать независимость внутренней и внешней мотивации (что практически всегда неверно).

Self-Supervision

Утверждение

В общем случае аналогичное разложение для $\mathcal{Z}^\pi$ неверно.

Доказательство:

это связано с тем, что внутренняя и внешняя мотивация могут быть скоррелированы, и этой информации при доступе к отдельным $\mathcal{Z}_{\text{extr}}^\pi$ и $\mathcal{Z}_{\text{intr}}^\pi$ у нас нет. Действительно: пусть известно, что для некоторых s, a с вероятностью 0.5 $\mathcal{Z}_{\text{extr}}^\pi(s, a)$ принимает значение +1, а иначе 0, и тоже самое для $\mathcal{Z}_{\text{intr}}^\pi(s, a)$. Мы могли бы сказать, что распределение суммы $\mathcal{Z}_{\text{extr}}^\pi(s, a) + \mathcal{Z}_{\text{intr}}^\pi(s, a)$ имеет такой вид: с вероятностью 0.25 мы получим +2, с вероятностью 0.5 получим +1, и с вероятностью 0.25 не получим ничего.

Self-Supervision

Продолжение доказательства:

однако, тогда мы предполагаем независимость этих случайных величин, а это может быть неправдой: например, вдруг мы получаем внешнюю $+1$ тогда и только тогда, когда получаем внутреннюю $+1$. Тогда $\mathcal{Z}^\pi(s, a)$ суммы мотиваций есть на самом деле $+2$ с вероятностью 0.5 и $+0$ иначе.



Следствие

В общем случае аналогичное разложение для $\mathcal{Z}^*$ неверно.

Self-Supervision

Разложение также позволяет использовать разные коэффициенты дисконтирования для разных мотиваций:

$$y^{\text{extr}} := r + \gamma^{\text{extr}} V_{\text{extr}}(s');$$

$$y^{\text{intr}} := r + \gamma^{\text{intr}} V_{\text{intr}}(s').$$

Разложение также позволяет во внутренней мотивации игнорировать понятия эпизодов, не используя флаги `done` из среды при обучении V^{intr} .

- Такой агент учитывает, что если он окажется в терминальном состоянии, то произойдёт сброс среды, и он окажется в стартовом состоянии $s_0 \sim p(s_0)$, для которого значение $V_{\text{intr}}(s)$ может быть высоким.
- Это может мотивировать агента прерывать текущий «неудачный» эпизод ради того, чтобы начать новый, что иногда может быть полезно.

Self-Supervision

На практике

В реальных задачах такое раздвоение MDP означает добавление к основной награде придуманной нами награды за исследование.

Вся сложность

Основная задача теперь создать хорошую функцию «внутренней мотивации» агента к исследованиям даже для тех сред, о которых нам самим не особо много известно.

Basic Idea

Самая простая идея

Начнем выдавать агенту дополнительную награду за посещение нового состояния среды. Данные награды будут пересоздаваться перед началом каждого эпизода. Однако такая функция наград поощряет не «умное» исследование, а просто исследование всего пространства состояний на каждом эпизоде.

Basic Idea

Самая простая идея

Начнем выдавать агенту дополнительную награду за посещение нового состояния среды. Данные награды будут пересоздаваться перед началом каждого эпизода. Однако такая функция наград поощряет не «умное» исследование, а просто исследование всего пространства состояний на каждом эпизоде.

Модификация (часто применяется на практике)

Не пересоздаём награды в посещенных ячейках между эпизодами, а заводим счетчики посещения состояний агентом за всё время обучения и, когда агент заходит в состояние, поощряем его обратно пропорционально счётчику (В теории такое решение делает *MDP* неприменимым, поскольку среда «подстраивается» под агента, но на практике всё хорошо).

Basic Idea

Пример

Пусть $h(s): \mathcal{S} \rightarrow \{0, 1 \dots N\}$ — некоторая хэш-функция состояний, называемая оракулом (oracle), и $n(i)$ — счётчик, сколько раз за время всего обучения нам встретились состояния с хэшем i . Тогда

$$r^{\text{intr}}(s, a) := \frac{1}{n(h(s)) + \varepsilon}, \quad \varepsilon > 0$$

называется нестационарным исследовательским бонусом; награда

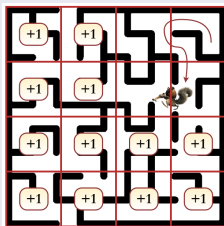
$$r^{\text{intr}}(s_t, a_t) := \mathbb{I}[\forall t' < t: h(s_t) \neq h(s_{t'})],$$

то есть награждение $+1$, если мы попали в состояние, хэш для которого $h(s_t)$ не встречался до этого в течение данного эпизода, называется эпизодичным исследовательским бонусом.

Basic Idea

Пример

В табличных MDP, где $|\mathcal{S}| < +\infty$, хэш-функция по сути не нужна: $h(s) = s$. В ряде сред подобный оракул можно придумать эвристически; например, если у агента есть понятие «координат», можно разделить пространство сеточкой, то есть поделив его на условные блоки, и награждать агента за «посещение большого числа блоков».



Недостатки самой простой идеи

- Данное решение невозможно применить в средах с непрерывным количеством состояний (среду придется бить на конечное число кусков и присваивать кускам награды; или нужно иметь некоторые дополнительные знания о среде, характерные только для неё).

Недостатки самой простой идеи

- Данное решение невозможно применить в средах с непрерывным количеством состояний (среду придется бить на конечное число кусков и присваивать кускам награды; или нужно иметь некоторые дополнительные знания о среде, характерные только для неё).
- Такой подход награждает все состояния одинаково (не выявляет важные или похожие друг на друга состояния).

Basic Idea

Недостатки самой простой идеи

- Данное решение невозможно применить в средах с непрерывным количеством состояний (среду придется бить на конечное число кусков и присваивать кускам награды; или нужно иметь некоторые дополнительные знания о среде, характерные только для неё).
- Такой подход награждает все состояния одинаково (не выявляет важные или похожие друг на друга состояния).
- Нужно помнить, в каких состояниях мы уже были.

Basic Idea

Mario Oracle

Пример хорошего *reward-shaping* на конкретном примере.

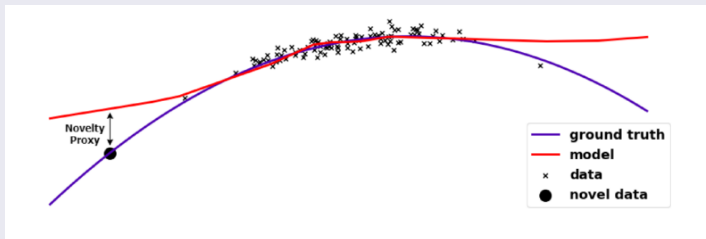


Будем поощрять движение агента вправо и наказывать за движение влево.

Random Network Distillation

Детектируем новые состояния

Рассмотрим некоторую задачу регрессии на картинке:



Задача обучения Q -функции — тоже задача регрессии! Поэтому новые состояния можно оценивать по этой задаче регрессии.

Random Network Distillation

Замечание

Обучать Q -функцию может быть трудно, особенно тяжело оценить насколько хорошо мы её выучили (иными словами — нужен таргет), а значит, смотреть на зазор ошибки Q -функции — плохая идея.

Другая задача регрессии!

Заведём вспомогательную нейронную сеть, выходы этой нейронной сети на наших состояниях и есть «новизна» состояний.

Random Network Distillation

Пусть $\phi: \mathcal{S} \rightarrow \mathbb{R}^d$ — некоторая функция, строящая эмбединги для состояний; например, случайная сеть (random network) — нейросеть со случайно инициализированными весами, которые не обучаются и никак не изменяются. Пусть $f: \mathcal{S} \rightarrow \mathbb{R}^d$ учится предсказывать выход ϕ , используя в качестве обучающей выборки её значения на встречавшихся в ходе обучения состояниях.

Определение

Задача обучения одной нейросети f на входах-выходах другой (заданной, фиксированной) нейросети ϕ

$$\mathbb{E}_s \|f(s) - \phi(s)\|_2^2 \rightarrow \min_f,$$

где математическое ожидание $\mathbb{E}_s$ берётся по произвольному буферу, называется дистилляцией (distillation).

Random Network Distillation

Так как ошибка ожидается выше там, где новее состояния, мы можем использовать это значение в качестве обучающего сигнала:

$$r^{\text{intr}}(s) := \|f(s) - \phi(s)\|_2^2.$$

Интуиция: если мы «впервые» увидели состояние s , модель f ещё ни разу не видела, какой эмбединг выдаёт на нём наша случайная сеть ϕ , и поэтому, скорее всего, ошибётся. Такой сигнал будет мотивировать агента отправиться в ту область среды, где обучающаяся нейросеть плохо предсказывает выход случайно проинициализированной нейросети.

Random Network Distillation

Недостатки такого метода

- Мы можем иметь большую ошибку не только на новых состояниях, которые модель ещё не видела, но и в том случае, если модель не обладает достаточной гибкостью.

Random Network Distillation

Недостатки такого метода

- Мы можем иметь большую ошибку не только на новых состояниях, которые модель ещё не видела, но и в том случае, если модель не обладает достаточной гибкостью.
- Мы можем столкнуться с большими проблемами, если целевые значения в задаче регрессии — случайные величины (недетерминированный таргет).

Random Network Distillation

Недостатки такого метода

- Мы можем иметь большую ошибку не только на новых состояниях, которые модель ещё не видела, но и в том случае, если модель не обладает достаточной гибкостью.
- Мы можем столкнуться с большими проблемами, если целевые значения в задаче регрессии — случайные величины (недетерминированный таргет).
- Ещё хуже, если в процессе обучения какие-то целевые значения меняются (нестационарный таргет).

Всё обсудили

Теперь перейдем к возможному варианту реализации проверки состояния на «новизну».

Random Network Distillation

Random Network Distillation (RND)

1. Заведем нейросеть, называемую *Target Network*, заинициализируем ей веса (случайным образом) и зафиксируем их навсегда.

Random Network Distillation

Random Network Distillation (RND)

1. Заведем нейросеть, называемую *Target Network*, заинициализируем ей веса (случайным образом) и зафиксируем их навсегда.
2. Заведем вторую нейросеть, называемую *Predictor Network*, которую уже будем обучать. Она будет пытаться спрогнозировать выход *Target Network* на состояниях среды.

Random Network Distillation

Random Network Distillation (RND)

1. Заведем нейросеть, называемую *Target Network*, инициализируем ей веса (случайным образом) и зафиксируем их навсегда.
2. Заведем вторую нейросеть, называемую *Predictor Network*, которую уже будем обучать. Она будет пытаться спрогнозировать выход *Target Network* на состояниях среды.
3. Невязку между значениями *Target Network* на данном состоянии и *Predictor Network* на данном состоянии будем интерпретировать как новизну этого состояния:

$$r^{\text{intr}}(s) := \frac{1}{2} \|\varphi^{\text{predictor}}(s) - \varphi^{\text{target}}(s)\|_2^2.$$

Random Network Distillation

Но есть нюансы!

- Случайная *Target Network* должна давать *различные* ответы на *различных* состояниях (аккуратная инициализация).

Random Network Distillation

Но есть нюансы!

- Случайная *Target Network* должна давать *различные* ответы на *различных* состояниях (аккуратная инициализация).
- Эмбединги состояний хорошо бы отнормировать, прежде чем давать их сетям. Для этого перед всем действием запускаем агента в среду, он собирает для нас состояния, по этому пакету считаем среднее и отклонение и далее используем их для нормировки при обучении (больше статистики не меняются — получим нестационарность таргета).

Random Network Distillation

Но есть нюансы!

- Первые несколько итерации обучения лучше вообще не использовать внешнюю награду, а пообучать внутреннюю (когда *Predictor* выйдет на асимптоту обучения).

Random Network Distillation

Но есть нюансы!

- Первые несколько итерации обучения лучше вообще не использовать внешнюю награду, а пообучать внутреннюю (когда *Predictor* выйдет на асимптоту обучения).
- Внутреннюю награду можно шкалировать по величине в зависимости от того, какие внутренние награды были на предыдущих шагах (скользящим оцениванием статистик по ходу обучения).

Curiosity

Любопытство

Идея: давайте смотреть не на «новизну» состояния, а на то насколько наш агент хорошо понимает как устроен мир. Иными словами давайте учить среду. $f(s, a)$ — функция, которая по текущим s, a предсказывает следующее состояние:

$$\frac{1}{2} \|f(s, a) - s'\|_2^2 \rightarrow \min_f.$$

Ошибка этой модели будем использовать как внутреннюю награду:

$$r^{\text{intr}}(s, a, s') := \frac{1}{2} \|f(s, a) - s'\|_2^2.$$

То есть нам неважно новое это состояние или нет; если мы можем предсказать, что будет дальше, то нам неинтересно.

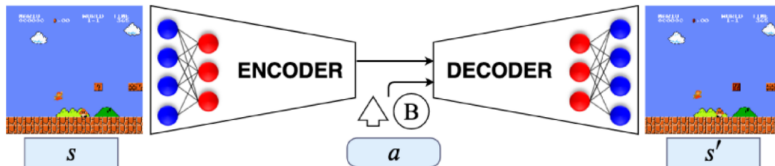
Curiosity

Определение

Любопытством (curiosity) называется ошибка модели мира агента.



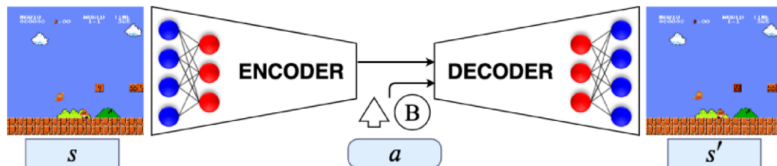
Curiosity: naive approach



Проблемы

- Генерация картинок — дорогой процесс.

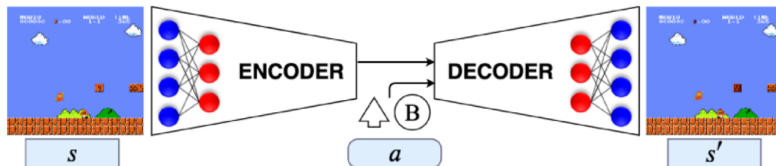
Curiosity: naive approach



Проблемы

- Генерация картинок — дорогой процесс.
- Выучиваем большое количество ненужной информации (выучиваем как движется вся сцена).

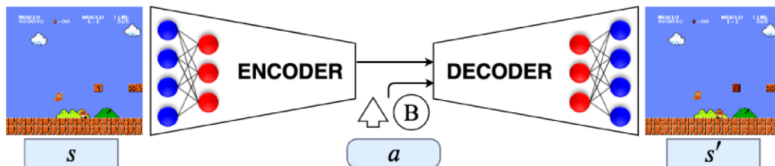
Curiosity: naive approach



Проблемы

- Генерация картинок — дорогой процесс.
- Выучиваем большое количество ненужной информации (выучиваем как двигается вся сцена).
- Учим модель в пространстве пикселей!

Curiosity: naive approach



Проблемы

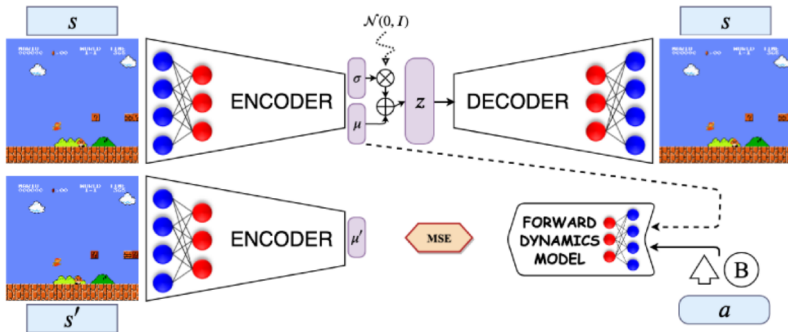
- Генерация картинок — дорогой процесс.
- Выучиваем большое количество ненужной информации (выучиваем как двигается вся сцена).
- Учим модель в пространстве пикселей!
- Что если среда не является детерминированной?

Curiosity: less naive approach

Идея

Возьмем *VAE* и обучим его на нашем буфере, зафиксируем его. Затем на каждом новом состоянии (картинке) *VAE* будет выдавать нам эмбединг этого состояния, который будет отправляться в *Forward Dynamics Model*. Таким образом мы получим *предсказание эмбединга следующего состояния*, остаётся только посчитать с помощью *VAE* эмбединг реального следующего состояния и вычислить *MSELoss* между двумя этими величинами.

Curiosity: less naive approach



Вывод

Перевод состояний в эмбединги это хорошо! Всё, что нам нужно — хорошие эмбединги :)

Noisy TV Problem



All you need is good embedding

Определение

Шумным телевизором (noisy TV) в среде называются принципиально непредсказуемые в силу стохастичности функции переходов явления.

«Проблема шумного телевизора»

Часто в средах на фоне происходят случайные бесполезные события, которые никак не помогают игроку достичь цели. Такие события для *RL*-агентов очень вредны, ведь на их восприятие тратится очень много ресурсов, поэтому нам хотелось бы, чтобы такие события отсекались на уровне эмбедингов. Но как?

Требования к эмбедингам

Фиксируем желания

Прежде чем решать главную проблему с шумом, давайте определим что такое «хороший» эмбединг по нашему мнению:

- не содержит бесполезного шума среды;

Требования к эмбедингам

Фиксируем желания

Прежде чем решать главную проблему с шумом, давайте определим что такое «хороший» эмбединг по нашему мнению:

- не содержит бесполезного шума среды;
- потенциально может содержать всю информацию, необходимую для оптимального решения задачи;

Требования к эмбедингам

Фиксируем желания

Прежде чем решать главную проблему с шумом, давайте определим что такое «хороший» эмбединг по нашему мнению:

- не содержит бесполезного шума среды;
- потенциально может содержать всю информацию, необходимую для оптимального решения задачи;
- желательно, чтобы латентное пространство эмбедингов было невелико и мы могли считать $MSELoss$;

Требования к эмбедингам

Фиксируем желания

Прежде чем решать главную проблему с шумом, давайте определим что такое «хороший» эмбединг по нашему мнению:

- не содержит бесполезного шума среды;
- потенциально может содержать всю информацию, необходимую для оптимального решения задачи;
- желательно, чтобы латентное пространство эмбедингов было невелико и мы могли считать $MSELoss$;
- эмбединги не должны меняться со временем или хотя бы меняться достаточно медленно (этот пункт мы выполнить не сможем).

Уменьшаем желания до выполнимых

Хороший эмбединг

Ищем функцию $\varphi(s) : S \rightarrow \mathbb{R}^d$ — перевода состояния в эмбединг такую, что:

- 1) не содержит бесполезного шума среды;
- 2) потенциально может содержать всю информацию, необходимую для оптимального решения задачи;
- 3) желательно, чтобы латентное пространство эмбедингов было невелико и мы могли считать *MSELoss*.

Inverse Dynamics Model

Определение

Модель, аппроксимирующая $p(a \mid s, s')$, называется моделью обратной динамики (inverse dynamics model).

Для модели обратной динамики проблема «шумных телевизоров» в среде заменяется симметричной проблемой в пространстве действий $\mathcal{A}$. Из формализма MDP по формуле Байеса следует:

$$p(a \mid s, s') = \frac{p(s' \mid s, a)\pi(a \mid s)}{\int_{\mathcal{A}} p(s' \mid s, a)\pi(a \mid s) da}.$$

Inverse Dynamics Model

Рассмотрим модель обратной динамики с «сиамской» архитектурой; для непрерывных пространств действий задача выглядит так:

$$\mathbb{E}_{s,a,s'} \left[\|g(\phi(s), \phi(s')) - a\|_2^2 \rightarrow \min_{g,\phi} \right],$$

а для классификации как

$$\mathbb{E}_{s,a,s'} \left[\ln g(a \mid \phi(s), \phi(s')) \rightarrow \max_{g,\phi} \right],$$

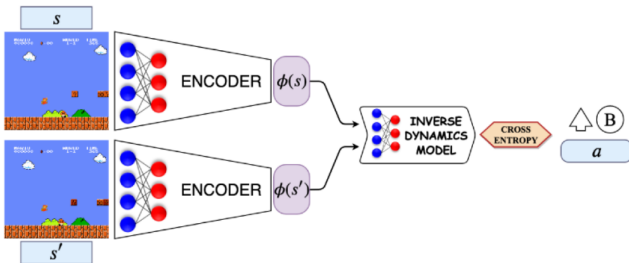
где $\phi: \mathcal{S} \rightarrow \mathbb{R}^d$ строит некоторое латентное описание состояний, а затем функция g пытается восстановить действие a , случившиеся между двумя состояниями, по их латентным описаниям.

Inverse Dynamics Model

Определение

Латентные представления состояний, которые учит функция $\phi(s)$ в задаче обучения обратной динамики, называются контролируемым состоянием (controlable state), а сама функция $\phi(s)$ – фильтром (filter).

Inverse Dynamics Model



Объяснение

Помочь достичь желаемое может модель, которая по двум соседним состояниям среды предсказывает действие между ними ($\phi(s)$ — тоже обучается).

Inverse Dynamics Model

На чём основан такой подход?

Для того, чтобы понять какое действие совершилось, фоновая шумовая информация нам не нужна, поэтому она отбрасывается сразу самой моделью.

Бонусы

На практике ещё часто к любопытству прибавляют невязку между предсказанным такой моделью действием и действительно выбранным в буфере (такие состояния кажутся более интересными, чем те, в которых предсказания модели и выбор агента совпали).

Inverse Dynamics Model

Замечание к бонусу

Невязка может быть большой не только потому, что состояние интересное, но и потому, что некоторые два различных действия из одного и того же состояния могут приводить в одно и то же состояние. Например, упор в стену в играх: стоять на месте и идти в сторону стены приводят к одному и тому же состоянию (упор в стену).

И это всё?

Inverse Dynamics Model — не единственный способ достичь желаемое.

Intrinsic Curiosity Model (ICM)

Идея

Сначала обучим эмбединги в *Inverse Dynamics Model*, а затем выкинем из неё блок под названием *Inverse Dynamics Model* (оставив только эмбединги). Вместо выкинутой части вставим модель прямой динамики и будем снова считать $MSELoss$ в латентном пространстве эмбедингов.

Вопрос

Не совсем понятно, где и когда учить *Inverse Dynamics Model* на самом первом шаге.

Решение

Давайте не будем выбрасывать *Inverse Dynamics Model*, а будем таскать её с собой и доучивать вместе с моделью прямой динамики.

Intrinsic Curiosity Model (ICM)

Свойство модели обратной динамики наводит на мысль, как можно построить защиту от шумных телевизоров. Для этого мы возьмём описание состояний и «почистим» их от шумных телевизоров при помощи фильтра $\phi(s)$, который переведёт описание состояния в некоторое латентное пространство, хранящее лишь частичную информацию о входе. А дальше модель прямой динамики построим в таком «отфильтрованном» латентном представлении:

$$\mathbb{E}_{s,a,s'} \|f(\phi(s), a) - \phi(s')\|_2^2 \rightarrow \min_f.$$

Итого модель ICM выглядит следующим образом (рассмотрим для примера случай непрерывных действий):

$$\mathbb{E}_{s,a,s'} [\|g(\phi(s), \phi(s')) - a\|_2^2 + \alpha \|f(\phi(s), a) - \phi(s')\|_2^2] \rightarrow \min_{f,g,\phi},$$

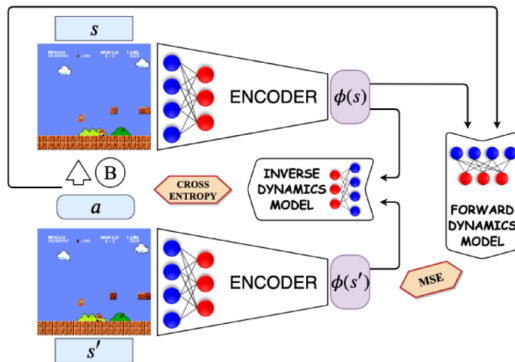
где s, a, s' — произвольные тройки из любого буфера, α — масштабирующий гиперпараметр.

Intrinsic Curiosity Model (ICM)

В качестве внутренней мотивации используется только ошибка модели прямой динамики:

$$r^{\text{intr}}(s, a, s') := \|f(\phi(s), a) - \phi(s')\|_2^2.$$

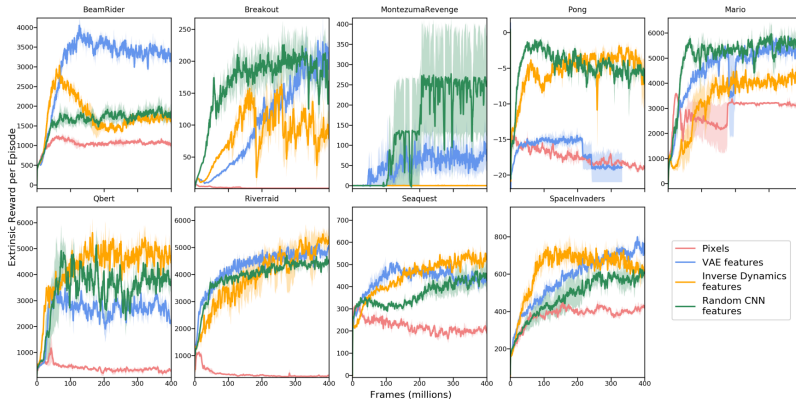
Intrinsic Curiosity Model (ICM)



Замечание

На этой схеме градиент затекает во все блоки! Появляется регуляризация на модель прямой динамики с помощью модели обратной динамики.

Сравнение



Large-Scale Study of Curiosity-Driven Learning (2018)

Наглядный пример

Curiosity: AI agents exploring without looking at any scores (video).

Телевизор и пульт — слабость *ISM* или нечто большее?

Проблема прокрастинации

ISM настроен фильтровать шум, неподконтрольный агенту, но если вдруг, агент может совершать действие и каждый раз получать неожиданный результат, то он *залипнет...*

Суть проблемы

Кажется, что данная уязвимость не привязана конкретно к алгоритму *ISM*, а является концептуально более важной задачей. Вдруг среда такова, что нужно долистать до сотового канала и остаться на нем? Вдруг случилось так, что мы не можем предсказать будущее не потому, что это нерелевантный шум, а потому, что наша предсказательная стратегия недостаточно настроена?

Телевизор и пульт — слабость *ИСМ* или нечто большее?

Вывод

По риторическим вопросам выше становится понятно, что управляемые шумные телевизоры придется оставить.